

Activity History: Week 1 - Python Basics and Control Flow

Developer: HMP

I. Environment Setup and Initial Configuration

The first step involves installing Python version 3.10 or later from the official python.org distribution. During installation, the PATH environment variable is configured to allow command-line access to the Python interpreter from any directory.

A code editor is selected for development. VS Code with the Python extension is chosen as the primary environment for its integrated terminal, syntax highlighting, and debugging capabilities. PyCharm Community Edition is an alternative offering a full IDE experience.

The installation is verified by executing 'python --version' in the terminal. A test file named 'hello.py' is created with a print statement. The file is executed via 'python hello.py', confirming the environment is operational.

CONFIGURATION COMPLETED

- Python 3.10+ installed with PATH configuration
- VS Code configured with Python extension
- First program executed: print("Hello, World!")

II. Variables and Data Types

Variables in Python store data without explicit type declarations. The interpreter determines the type at runtime. Variable names must start with a letter or underscore, can contain alphanumeric characters, and are case-sensitive. The snake_case naming convention is standard in Python.

Five primary data types are covered: integers (int) for whole numbers, floats (float) for decimal values, strings (str) for text data enclosed in quotes, booleans (bool) for True/False values, and None for representing absence of value. The type() function returns the class of any object.

String operations include concatenation (+), repetition (*), and length measurement (len()). Multi-line strings use triple quotes. Underscores improve numeric literal readability (e.g., 1_000_000).

DATA TYPES COVERED

- int: whole numbers (42, -10, 1_000_000)
- float: decimal numbers (19.99, 3.14159, 2.5e6)
- str: text with single, double, or triple quotes
- bool: True or False values / None: absence of value

III. Input/Output Operations and Operators

The `print()` function outputs data to the console. It accepts multiple arguments and two optional parameters: `'sep'` defines the separator between values (default is space), `'end'` defines the line terminator (default is newline). F-strings provide formatted output: `f"text {variable}"`.

The `input()` function captures user input and returns a string. The `int()` and `float()` functions convert text input to numeric types. Validation is necessary since `input()` always returns a string.

Three categories of operators are covered. Arithmetic: `+`, `-`, `*`, `/`, `//` (floor division), `%` (modulo), `**` (power). Comparison: `==`, `!=`, `<`, `>`, `<=`, `>=` (return booleans). Logical: `and`, `or`, `not` (combine boolean expressions).

OPERATORS REFERENCE

- Arithmetic: `+`, `-`, `*`, `/`, `//`, `%`, `**`
- Comparison: `==`, `!=`, `<`, `>`, `<=`, `>=`
- Logical: `and`, `or`, `not`

IV. Control Flow Structures and Applied Project

Conditional statements control program execution based on boolean expressions. The `if` statement executes a block when its condition is `True`. The `else` clause provides an alternative when the condition is `False`. The `elif` keyword chains multiple conditions in sequence.

Python uses indentation to define code blocks. Consistent indentation of 4 spaces per level is required. Conditions can be combined using logical operators (`and`, `or`) and nested within other conditionals.

The week concludes with a grade classifier project. The program receives a numeric score via `input()`, converts it to an integer, then uses chained `if/elif/else` statements to assign a letter grade (A, B, C, D, or F) based on defined thresholds. The result is displayed using an f-string with both the score and corresponding grade.

PROJECT IMPLEMENTATION

- `input()` receives the numeric score as string
- `int()` converts the string to an integer value
- `if/elif/else` evaluates thresholds and assigns grade
- f-string outputs the score and letter grade